Building data course — Week 2 — units, joins and conditions

sparql-playground / lessons

Generated for print and offline reading

Week 2 — units, joins and conditions

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. The scripts are in the repo next to each lesson. Keep notes under the ignored `student-work/` folder.

Contents

- Day 6 — Units and SI: convert once, then do all fault math in °C, Pa and fractions
- Day 7 — Two AHUs: stack them, then join them on time
- Day 8 — CASE and conditional counts: run hours, unoccupied runtime, percent occupied
- Day 9 — Time buckets: hourly and daily averages
- Day 10 — Open-FDD events: episodes per rule, then which rows fall inside an episode

---

Day 6 — Units and SI: convert once, then do all fault math in °C, Pa and fractions

Goal: turn the BAS's US-unit columns into SI (°C, pascals, fractions 0–1) in a few lines of pandas and one SQL query, and avoid the one conversion mistake everybody makes: converting a temperature *difference*.

Read

The CSV is what the building automation system (BAS) logged: temperatures in °F, pressures in inches of water (inH2O), valve and fan speeds in percent 0–100. **Open-FDD's fault math is SI-only**: °C (or K for differences), pascals, and fractions 0–1. There are two reasons:

1. **One set of thresholds.** Rules ship with numbers like "25 Pa" or "2 K". If every site could feed °F one day and °C the next, every rule would need a unit switch. Instead the *ingest* step converts, and the rules never see US units.
2. **Fewer silent bugs.** A rule that compares `percent` to `0.95` is wrong by 100× and never throws an error. If everything is a fraction, "95%" is `0.95` everywhere.

So the first step of every rule is a conversion. Every script from here on starts with it.

Quantity	CSV unit → SI	Formula
temperature (absolute)	°F → °C	`(F - 32) * 5 / 9`
temperature (difference)	°F → K	`F * 5 / 9` (**no 32**)
static pressure	inH2O → Pa	`inwc * 249.08891`

Quantity	CSV unit → SI	Formula
valve, damper, fan speed	percent → fraction	`pct / 100`

The delta trap. The 32 in `(F - 32) * 5 / 9` exists because 0 °F and 0 °C are different *places* on the scale. A *difference* has no zero point to shift: a 5 °F rise is a 5 × 5/9 = **2.78 K** rise. Push it through the absolute formula and you get `(5 - 32) * 5 / 9 = -15`, a cooling of 15 degrees. It's a typo-sized mistake that turns a "DAT is 5 °F over setpoint" check into nonsense. The scripts average the discharge-air-temperature error (discharge air minus its setpoint) both ways, the right way and the wrong way.

Conversions of this kind are linear, so the mean of the converted column equals the converted mean. That's a cheap sanity check, but only for linear conversions.

In pandas each conversion is one line that makes a new column (`df["outside_air_temp_c"] = (df["outside_air_temp_f"] - 32) * 5 / 9`). In SQL the same arithmetic goes inside `AVG(...)`. Watch the `/ 9.0` and `/ 100.0` in SQL: SQLite divides integers as integers (`5 / 9` is `0`), so one number must be a float. pandas always does true division.

Run

```
uv run python lessons/course.py run 6
```

That runs `day06_pandas.py`, then `day06_sql.py`. Both print:

```
Average discharge air temperature (C):
16.206
Average outside air temperature (C):
16.265
Average duct static pressure (Pa):
195.035
Average supply fan speed (fraction):
0.39
Average DAT error, right (K):
6.782
Average DAT error, wrong (K):
-10.996
```

The right and wrong DAT errors differ by 17.8 K, which is 32 × 5/9: the shift the absolute formula adds.

A SQL *view* (`CREATE VIEW ahu_1_si AS SELECT ...`) would store these conversions once so every later query could reuse them; Day 9's Read section says more about views. Here one query is enough.

Try it

1. Add the mixed-air temperature, `mixed_air_temp_f` → °C, to both scripts. Is the mean what you expect next to the discharge and outside air means? (In pandas, add the column to `columns` first.)
2. A rule says "fault if discharge air is more than 5 K above its setpoint". Count the rows where the right DAT error is above 5 K and where the wrong one is. In pandas: `(dat_error_k > 5).sum()`; in SQL: `SUM((discharge_air_temp_f - dat_reset_f) * 5 / 9.0 > 5)`. And if the threshold were written as 9 °F, what do you compare against?

Answers

1. SQL: `AVG((mixed_air_temp_f - 32) * 5 / 9.0)`; pandas: `df["mixed_air_temp_c"] = (df["mixed_air_temp_f"] - 32) * 5 / 9`.
2. The right error is above 5 K in 16,488 rows (about 46% of the history). The wrong one only in 2,058, because it is shifted by about −17.8 K. A 9 °F threshold is itself a *difference*, so convert it as `9 * 5 / 9 = 5 K`: multiply, don't subtract 32.

Why it matters

Every fault threshold from here on is written in SI, so every script from here on starts with the conversion you wrote today. If the conversion is wrong, every fault count downstream is wrong in a way no test on the *rule* will catch. Week 5 moves the unit into the data model (each point says what unit it logs), so a rule can ask for “duct static in Pa” and the conversion is looked up, not typed.

Day 7 — Two AHUs: stack them, then join them on time

Goal: put both AHUs in one table (stack), then join them on `timestamp_utc` to ask "how far apart are two sensors that measure the same weather?", in pandas and SQL.

Read

Building 50 has two air handlers, AHU_1 and AHU_2, each logged in its own CSV (and its own SQLite table) with the same column names. Two different questions need two different tools.

Stack: "give me all the rows." One table, AHU_2's rows below AHU_1's, with an `equipment_id` column saying who is who. This is `UNION ALL` in SQL and `pd.concat` in pandas (add the id column to each piece first). Use `UNION ALL`, not `UNION`: plain `UNION` also removes duplicate rows, which costs a sort and would silently drop legitimate repeats. The stacked table has 71,072 rows, twice 35,536.

Join: "put the two rows for the same time side by side." One row per timestamp, one column per AHU. Both AHUs sit in the same building and see the same weather, so their outside-air temperatures should nearly agree. A big gap means one sensor is wrong, a better data-quality check than any threshold on a single sensor. The difference of two temperatures is a *difference* (Day 6), so it converts with `* 5 / 9`, no 32.

Building 50's answer: on average the sensors differ by 1.64 K, 9,005 of 35,536 rows by more than 2 K, and 7 rows by more than 20 K. Those 7 are AHU_2 reading exactly 0 °F, a sensor dropout.

Step	`day07_sql.py`	`day07_pandas.py`
stack	`SELECT ... FROM ahu_1 UNION ALL SELECT ... FROM ahu_2`	`pd.concat([ahu_1, ahu_2])`
join on time	`FROM ahu_1 AS a JOIN ahu_2 AS b ON b.timestamp_utc = a.timestamp_utc`	`ahu_1.merge(ahu_2, on="timestamp_utc")`

Step	`day07_sql.py`	`day07_pandas.py`
count rows over a gap	`SUM(diff_k > 2)`	`(diff_k > 2).sum()`

How a join runs (for the curious). For each AHU_1 row the database finds the AHU_2 row with the same timestamp. `timestamp_utc` is the primary key of each SQLite table, so SQLite looks rows up in an index (cost about n log m). pandas builds a hash table of one side and probes it (cost about n + m). Both are fast; a naive double loop that compares every pair (n × m) is not, which you can see by timing 1,500 × 1,500 timestamps in plain Python.

Join explosion. The join above works because `timestamp_utc` is unique in each table. If the join key repeats, every left row pairs with *every* matching right row. Join the first day of the two AHUs on the **hour** (12 rows per hour) instead of the timestamp and 280 rows per side become 3,328 rows. Add up a `SUM` or `COUNT` over a join like that and every number is silently inflated. pandas has a guard: `merge(..., validate="1:1")` raises `MergeError` if either side's key repeats. SQL has no such keyword; check `COUNT(*) > COUNT(DISTINCT key)` before you trust a join.

Run

```
uv run python lessons/course.py run 7
```

That runs `day07_pandas.py`, then `day07_sql.py`. Both print:

```
Rows in the stacked table: 71072
Rows in the joined table:  35536
Average gap between the sensors (K): 1.64
Rows more than 2 K apart:  9005
Rows more than 20 K apart: 7
```

Try it

1. Change the 2 K threshold to 1 K in both scripts. How many rows are more than 1 K apart?
2. Make it a left join (`LEFT JOIN` / `how="left"`) after deleting one AHU_2 row (in pandas: `ahu_2 = ahu_2.drop(ahu_2.index[100])`). What is the AHU_2 value on the missing timestamp, and how does that differ from the inner join's result?
3. Join explosion: in pandas, add `ahu_1["hour"] = ahu_1["timestamp_utc"].str[:13]` (and the same for `ahu_2`), then `ahu_1.merge(ahu_2, on="hour")`. How many rows come back? Now add `validate="1:1"` and see the error.

Answers

1. Compare `diff_k > 1`: 17,215 rows, just under half. The two sensors disagree by more than 1 K for much of the record, so a rule that mixes them shouldn't assume they match.
2. The left join keeps the AHU_1 row and fills the AHU_2 value with `NaN` (SQL: `NULL`); the inner join silently drops the row, so you would never know a timestamp was missing. Left joins are how you *find* gaps.
3. 35,536 rows per AHU become about 12 times that: each hour repeats 12 times per AHU, so each row matches 12 rows on the other side. With `validate="1:1"` pandas raises `MergeError`.

Why it matters

Stacking and joining are the two shapes of nearly every analysis: "same question for every AHU" and "relate two measurements at the same time". Both are exact and cheap *when the key is unique and you know it is*. The join explosion is the classic way an energy or fault number comes out 12× too big and nobody notices. Day 8 adds `CASE`, so one pass over the rows can answer several conditional questions at once.

Day 8 — CASE and conditional counts: run hours, unoccupied runtime, percent occupied

Goal: ask conditional questions ("static pressure *while the fan runs*", "run hours *while unoccupied*") with `CASE` in SQL and True/False conditions in pandas.

Read

A fan that is off has no duct static pressure. If you average `da_p_inwc` over all rows you mix "the fan was off" into "the pressure was low", and the number means nothing. Almost every fault rule starts by **gating**: judge only the rows where the equipment is doing its job. That needs a condition inside the query.

In pandas a condition is a column of True/False, one per row: `fan_on = df["supply_fan_status"] == 1`. Combine conditions with `&` (and), `|` (or) and `~` (not). Then:

- `fan_on.sum()` counts the True rows (True counts as 1),

- `fan_on.mean()` is the fraction of True rows,
- `df[fan_on]` keeps only those rows, so `df[fan_on]["da_p_inwc"].mean()` averages only while the fan runs.

In SQL the tool is **`CASE`**, SQL's if/else for one row: `CASE WHEN fan = 1 THEN 1 ELSE 0 END`. The first `WHEN` that matches wins; `ELSE` is the default (leave it out and you get `NULL`). Put it inside an aggregate and you get **conditional aggregation**:

- `SUM(CASE WHEN condition THEN 1 ELSE 0 END)` counts the rows where the condition holds,
- `AVG(CASE WHEN condition THEN value END)` averages only those rows, because `AVG` skips the `NULL`s that a `CASE` without an `ELSE` produces.

Many different counts come out of one pass over the table: total rows, running rows, rows that run while unoccupied, occupied rows.

Idea	`day08_sql.py`	`day08_pandas.py`
a condition	`supply_fan_status = 1`	`fan_on = df["supply_fan_status"] == 1`
combine	`AND`, `OR`, `NOT`	`&`, `|`, `~`

Idea	day08_sql.py	day08_pandas.py
count rows	`SUM(CASE WHEN c THEN 1 ELSE 0 END)`	`c.sum()`
fraction of rows	`100.0 * SUM(...) / COUNT(*)`	`c.mean() * 100`
average only some rows	`AVG(CASE WHEN c THEN x END)`	`df[c]["x"].mean()`

Careful with `<>` and `NULL`: `occ_c <> 1` is `NULL`, not true, when `occ_c` is `NULL`. This data has no NULLs, but a real export will; a `CASE` or a condition is where they quietly drop rows.

What AHU_1 says: it ran 1,574.42 hours, of which 58.58 hours were while unoccupied (3.72% of its run time), and it was scheduled occupied 51.26% of the time. `occ_c` is the BAS's occupancy command (1 occupied, 0 unoccupied). Some unoccupied running is normal (morning warm-up, night purge) and some is a schedule override someone forgot about. That ratio is exactly what an "AHU running while unoccupied" rule would look at.

Run

```
uv run python lessons/course.py run 8
```

That runs `day08_pandas.py`, then `day08_sql.py`. Both print:

```
Run hours: 1574.42
Run hours while unoccupied: 58.58
Percent of rows occupied: 51.26
Percent of run time unoccupied: 3.72
Average static while the fan runs (Pa):
335.6
```

Each row is 5 minutes, so rows × 5 / 60 = hours. Averaged over *all* rows the static pressure is only 195 Pa (Day 6), because the fan is off nearly half the time; while it runs it is 335.6 Pa.

Try it

1. Add the average static pressure while the fan is *off* (`supply_fan_status <> 1`) to both scripts. What do you expect, and what do you get?
2. Count the rows where the fan is running but static pressure is below 50 Pa ("running with no pressure"). How many of those are occupied?
3. Speed (pandas only): the same classification written row by row with `df.apply(my_function, axis=1)` is far slower than a True/False column: on 71,072 rows roughly half a second against a few milliseconds. In pandas, reach for

conditions first. SQL is already "vectorized": the engine does the loop, and you describe the condition.

Answers

1. SQL: `AVG(CASE WHEN supply_fan_status <> 1 THEN da_p_inwc * :pa_per_inh2o END)`; pandas: `df[~fan_on]["da_p_inwc"].mean() * PA_PER_INH2O`. About 35.5 Pa: the fan is off, so there is almost no pressure.
2. SQL: `SUM(CASE WHEN supply_fan_status = 1 AND da_p_inwc * 249.08891 < 50 THEN 1 ELSE 0 END)`; pandas: `(fan_on & (df["da_p_inwc"] * PA_PER_INH2O < 50)).sum()`. AHU_1: 460 rows, 209 of them occupied. A fan that "runs" with no pressure is a failed fan or a status point that lies, the kind of thing Day 14's FC1 rule flags.

Why it matters

"While running", "while occupied", "while unoccupied" are the words every fault rule starts with. In Day 14 they become the gate `supply_fan_status = 1`, and in later weeks they come from the data model instead of a hard-coded column. Conditional aggregation is also how you build a dashboard row (hours run,

hours faulted, percent occupied) in one pass over the data. Day 9 groups the same rows by time instead of by condition.

Day 9 — Time buckets: hourly and daily averages

Goal: collapse 5-minute rows into hourly and daily averages in pandas and SQL: give every row a bucket (the hour or day it belongs to), then `GROUP BY` the bucket.

Read

A year of 5-minute data is 105,000 rows per point. A dashboard, a report or a daily-energy number wants hours or days. **Bucketing** means: give every row a key for "which hour (or day) am I in", then group by that key and average. Only the key changes; the rest is the grouping you already know.

The key is the timestamp written out as text with only the parts you want: `2026-03-16 00` for the hour, `2026-03-16` for the day.

Step	`day09_sql.py`	`day09_pandas.py`
hour key	`strftime('%Y-%m-%d %H', timestamp_utc)`	`df["time"].dt.strftime("%Y-%m-%d %H")`
day key	`strftime('%Y-%m-%d',`	`df["time"].dt.strftime("%Y`

Step	day09_sql.py	day09_pandas.py
	`timestamp_utc)`	`-%m-%d")`
average per bucket	`GROUP BY hour` + `AVG(...)`	`df.groupby("hour").agg(...)`

In pandas the CSV timestamp is text, so `pd.to_datetime` first turns it into a real timestamp (the `.dt` tools need that). SQLite's `strftime` reads the text directly.

A bucket with few rows is a partial bucket: the record starts at 00:40, so the first hour has only **4 rows** instead of 12. A rule that needs "12 samples per hour" has to know about partial buckets at the edges. That is why the scripts keep a `rows` count next to each average.

AHU_1 has 2,962 hourly rows and 124 daily rows. Both scripts group on text, so an hour with no data simply doesn't appear; Building 50 has no gaps, but a real building will have some.

Other ways to say the same thing (not in the scripts):

- pandas: `df["time"].dt.floor("h")` rounds a timestamp down to its hour and keeps it a real timestamp; and `df.set_index("time")`

`["col"].resample("h").mean()` does the bucketing and the average in one call, adding a `NaN` row for an hour with no data.
- SQL: `unixepoch(timestamp_utc) / 3600` is the number of hours since 1970. Integer division makes a number that is cheap to compare and easy to index; `strftime` makes text that is easy to read.

Run

```
uv run python lessons/course.py run 9
```

That runs `day09_pandas.py`, then `day09_sql.py`. Both print:

```
Hourly rows: 2962
Daily rows:  124
First hour: 2026-03-16 00 has only 4 rows
(a partial hour)
Hottest day: 2026-07-01 at 31.3 C
```

Try it

1. Which day has the *lowest* mean outside-air temperature? (Sort the other way: `.iloc[0]` in pandas, `ORDER BY mean_oa_c LIMIT 1` in SQL.)
2. Change the hourly buckets to half-days in both scripts: before 12:00 and after. What do you change in the key?

3. **View vs summary table (SQL).** Once you have an hourly query there are two ways to keep it: `CREATE VIEW hourly_v AS SELECT ...` stores the **query**, so every read re-runs the aggregation (it can't go stale, and costs no storage), while `CREATE TABLE hourly_t AS SELECT ...` stores the **result**: computed once, reads are instant, but it is a snapshot that must be rebuilt when new rows arrive. Try both and time ten reads of each with `time.perf_counter()`: the table is hundreds of times faster. It is the same trade-off as caching anywhere, and what Timescale calls a *continuous aggregate*.

Answers

1. 2026-03-17 at −5.9 °C. The record opens with a flat, stuck-looking reading of 20.6 °C: a reminder to look at the data before trusting a mean.
2. Use the first 10 characters plus the AM/PM: in pandas `df["time"].dt.strftime("%Y-%m-%d %p")`, in SQL `strftime('%Y-%m-%d %p', ...)` is not available, so use `substr(timestamp_utc, 1, 10) || CASE WHEN CAST(strftime('%H', timestamp_utc) AS INTEGER) < 12 THEN ' AM' ELSE ' PM' END`.

Why it matters

Hourly and daily summaries are what people read, and what slower rules (daily energy, hours outside setpoint) run on. Bucketing is also where timezones bite: these buckets are UTC, so a “day” starts in the evening in US time zones. A report for building staff has to bucket in local time. And the view/table choice is the first taste of a decision you’ll make repeatedly: derive on read, or store once. Day 10 does a different kind of time matching: finding which fault episode each row falls in.

Day 10 — Open-FDD events: episodes per rule, then which rows fall inside an episode

Goal: count Open-FDD's fault episodes and hours per rule, then mark which 5-minute history rows fall inside an episode, in pandas (a short loop) and SQL (`BETWEEN` with `EXISTS`). Then finish Week 2 with a challenge that uses everything.

Read

Open-FDD writes its findings as **episodes**: one row per stretch of time a rule was in fault, with a start, an end and the rule's name. They ship next to the CSVs in `data/BUILDING_50/*/fdd_events.csv`, and the SQL side has them as the `fdd_events` table (columns `start_time_utc`, `end_time_utc`, `hvac_system`, `rule_name`, `duration_min`, `sample_count`). `hvac_system` is `bld_50_AHU_1` for AHU_1. Counting is a plain `GROUP BY`:

```
          rule_name  episodes  hours
economizer_not_used         1    0.0
                fc1        80  484.1
               fc13        90  510.0
      sensor_bounds         2    0.0
```

That answers "how many, how long", but not "what was the AHU doing during them?" For that, the history rows and the episodes need to meet: the history has one row per instant, the episodes one row per *interval*. A row is **inside** an episode when `start <= timestamp <= end`.

The beginner approach is the one Day 14 used: **take the episodes one at a time**. For each episode, find the history rows between its start and end and mark them. The timestamps are text in one format (`2026-07-15 22:44:01+00:00`), so comparing the text compares the times.

Step	`day10_sql.py`	`day10_pandas.py`
episodes per rule	`GROUP BY rule_name`	`events.groupby("rule_name").agg(...)`
row inside an episode	`h.timestamp_utc BETWEEN e.start_time_utc AND e.end_time_utc`	`(ts >= start) & (ts <= end)`

Step	day10_sql.py	day10_pandas.py
mark / count the rows	`WHERE EXISTS (SELECT 1 FROM fdd_events AS e WHERE ...)`	`for _, event in fc13_events.iterrows(): then df.loc[inside, "in_episode"] = True`

`EXISTS` asks “is there at least one episode around this row?”, so a row inside several overlapping episodes still counts once. The pandas loop gets the same answer: a row is marked True however many episodes mark it.

Overlap. Open-FDD’s episodes overlap heavily: AHU_1’s `fc13` has 90 episodes. That is why the summed `hours` above (510 h) is so much larger than the time the rows are actually covered (1,206 rows × 5 min = 100.5 h): the episodes overlap and the sum counts the overlap many times. Count rows (or merge the intervals), don’t add up `duration_min` when intervals overlap.

Faster ways (for later). The loop does 90 passes over 35,536 rows, which is instant here. With thousands of episodes you would use a sorted search (`np.searchsorted`) or `pd.merge_asof` (“the latest episode that started

before this row"), and in SQL an index on the episode start time. `merge_asof` returns one match per row, so with overlapping episodes it can miss a row that an earlier, longer episode still covers.

Run

```
uv run python lessons/course.py run 10
```

That runs `day10_pandas.py`, then `day10_sql.py`. Both print:

```
          rule_name  episodes  hours
economizer_not_used         1    0.0
                fc1        80  484.1
               fc13        90  510.0
      sensor_bounds         2    0.0
fc13 rows inside an episode: 1206
Hours those rows cover (5 min each): 100.5
Hours if you add up every episode:   510.0
(episodes overlap)
```

Try it

1. Change `"fc13"` to `"fc1"` in both scripts. How many rows are inside an fc1 episode, and how does the covered time compare with the 484.1 hours you get by adding up the episodes?
2. AHU_2 has no `fc1` episodes at all. Point the scripts at AHU_2 (`AHU_2` in the CSV

path, `bld_50_AHU_2` in the SQL) and use `"fc1"`. What do you get, and why does it matter that SQL's `COUNT(*)` gives `0` rather than no rows?

3. **Week 2 challenge.** Add `supply_fan_status` and `outside_air_temp_f` to the history load. Split AHU_1's rows into *inside an fc13 episode* and *outside*, and report the row count, the percent of rows with the fan running, and the mean outside-air temperature in °C for each group. What does it say about when fc13 fires?

Answers

1. Change the rule name and run both scripts; they still agree. The rows inside fc1 episodes cover far less time than 484.1 hours, because those episodes overlap too.

2. Zero rows inside an episode. A missing group is a join's most common silent failure: with an `INNER JOIN` AHU_2 would vanish from a report, which would look as though AHU_2 wasn't analyzed rather than healthy. `EXISTS` and the `False` column keep every row.

3. Result (both languages agree):

```
in_episode   rows  pct_fan_running
mean_oa_c
     False  34330             51.5
15.9
      True   1206            100.0
26.3
```

Every row inside an `fc13` episode has the fan running (100%), and it's warm outside: 26.3 °C against 15.9 °C otherwise. The rule fires when the AHU is working in warm weather, which is when you'd expect a cooling-related rule to find trouble. In pandas: `df.groupby("in_episode")` with `agg(rows=("in_episode", "size"), fan=("supply_fan_status", "mean"), oa_f=("outside_air_temp_f", "mean"))`, then convert the mean temperature with Day 6's formula. In SQL: select `EXISTS (...)` as `in_episode` and `GROUP BY in_episode`. This uses Day 6 (units), Day 8 (conditions) and Day 9 (grouping) with today's episode matching.

Why it matters

Open-FDD tells you *that* a rule fired; matching rows to episodes is how you ask *what the equipment was doing* when it did: the fan state, the weather, the setpoint, the schedule. Day 14 already compared your own

rule with Open-FDD's episodes this way, and later weeks do it again. Watch for the two traps from this week that bite here too: a join on a repeated key multiplies rows (Day 7), and summing episode durations double counts where episodes overlap.

Reading — Week 2

The lessons first, then these for exact behavior.

- SQLite joins (SELECT, the FROM clause)
- SQLite CASE expressions
- SQLite CREATE VIEW
- DB Browser: importing CSV
- TimescaleDB time_bucket